

Project Part B

1 Introduction

Our Cascade agent uses a variety of techniques that improve its performance. A depth-limited minimax search forms the foundation of our agent with a finely tuned evaluation function, with different weights for the opening, midgame, and endgame. To make this search practical under the referee's time and memory limits, the agent uses α - β pruning, iterative deepening, dynamic time control, tactical action ordering, and fixed root/internal action limits. Additionally, to improve efficiency, we created a custom `MyBoard` class to store the board and implemented an incremental Zobrist hash.

2 Action Selection

At the beginning of each search, the agent first gets all the legal actions from the current state of the board. During the placement phase, it checks valid cells. During the play phase, it gets all possible MOVE, EAT, CASCADE actions.

Then the agent sorts the actions according to priority score.

Action	Priority score
EAT	$400 + 40h_t$
CASCADE	$180 + p_{cas}$
MOVE to friendly stack	$140 + 10h_d$
MOVE to empty cell	$60 - 4d_c$
PLACE	$80 - 8d_c$
Invalid or unsupported action	0

Table 1: Priority scores used for action ordering

Here, h_t is the height of the captured enemy stack, p_{cas} is the directional cascade potential, h_d is the height of the destination friendly stack, and d_c is the destination or placement cell's distance from the board centre.

The agent also combines priority-based action ordering with iterative deepening. When a search at a depth is completed, the best result is pushed to the first so that it is searched first at the next depth.

3 Search Algorithm

3.1 Minimax with alpha-beta pruning

The core algorithm used in the project is minimax with α - β pruning. It is used because both players have complete information and actions are deterministic.

Mathematically, we can define below function: For a state s , depth d , transition function T , and evaluation function E , the searched value is:

$$V(s, d, \alpha, \beta) = \begin{cases} E(s), & d = 0 \text{ or } s \text{ is terminal,} \\ \max_{a \in A(s)} V(T(s, a), d - 1, \alpha, \beta), & \text{our turn,} \\ \min_{a \in A(s)} V(T(s, a), d - 1, \alpha, \beta), & \text{opponent's turn.} \end{cases}$$

3.2 Dynamic depth and time control

The searching time of Minimax heavily relies on the search depth, so we use a dynamic depth limit to improve the agent’s efficiency.

Condition checked in order	Search depth
Remaining time ≤ 2 seconds	1
Remaining time ≤ 6 seconds	2
Placement phase	2
Remaining play turns ≤ 20 and actions ≤ 18	4
Actions ≤ 10 and remaining time > 20 seconds	5
Actions ≤ 10 and remaining time ≤ 20 seconds	4
Default play phase	3

Table 2: Dynamic search depth limit. Earlier rows have priority over later rows

Each call to `action` receives a dynamic CPU budget based on the referee’s reported remaining time, the current phase, the branching factor, and the estimated number of future turns. The search checks the deadline throughout the tree and raises an internal timeout when needed.

3.3 Localized board representation

During the search, the agent works on a local `MyBoard` rather than the referee `Board`. The position is stored as a flat `numpy.int8` array of 64 cells: a Red stack of height h is encoded as $+h$, a Blue stack as $-h$, and an empty cell as 0. This compact layout reduces iteration overhead, and the Zobrist hash described in Section 3.4 is maintained incrementally on every `apply_action` and `undo_action`. The agent also keeps a referee `Board` in parallel to do final legality checks on the action returned each turn.

3.4 Transposition table

The agent uses a persistent transposition table to avoid re-searching previously visited positions. Each entry maps a Zobrist-hashed board state to (`depth`, `flag`, `score`, `best_action`). A cached result is reused directly if the stored depth is sufficient; otherwise, the stored best action is still used to seed move ordering, which improves $\alpha - \beta$ pruning at the new depth. The table uses depth-preferred replacement so that a shallower entry never overwrites a deeper one.

3.5 Killer-move heuristic

The killer-move heuristic stores quiet moves that previously caused a β -cutoff during search. These moves are likely to refute other branches at the same search depth, so they are prioritized in later searches. Tactical actions such as Eat and Cascade are not stored

because they already receive high priority from the normal action ordering. During move ordering, the search first considers the transposition-table best move, then killer moves, followed by the remaining actions.

4 Evaluation Function

4.1 Main formula

$$E(s) = w_T^{(p)} \Delta T + w_S^{(p)} \Delta S + w_E^{(p)} \Delta E + w_C^{(p)} \Delta C + 15\Delta H + S_P + S_T + S_R$$

Symbol	Description
ΔT	Token count difference between the agent and the opponent
ΔS	Stack count difference
ΔE	Difference in available eat actions
ΔC	Difference in cascade potential
ΔH	Threat score difference
S_P	Placement score
S_T	Turn limit score
S_R	Repetition score
p	Current game phase
$w^{(p)}$	Phase-dependent weights

Table 3: Notation used in the evaluation function

4.2 Coefficient Tuning

The coefficients in $E(s)$ were chosen in three stages. First, we set initial values based on the strategic role of each feature, with material weighted highest and positional features one to two orders of magnitude smaller. Second, we tuned them through match-play against three reference opponents described in Section 5.1. Third, we extended the evaluation function with two refinements: phase-aware weighting and a threat differential term.

4.2.1 Phase-Aware Weights

In the opening, the board is sparse and merging stacks can significantly reshape the position. In the endgame, the turn-limit tiebreaker is decided by raw token count, so material advantages must be preserved and converted decisively. Therefore, the agent selects between three weight sets according to the remaining number of play-phase turns.

Phase	Remaining turns	w_m	w_s	w_e	w_c
Opening	> 200	1000	60	100	3
Midgame	100–200	1000	40	120	2
Endgame	≤ 100	1600	25	130	2

Table 4: Phase-aware evaluation weights

The higher opening stack weight encourages friendly merges, which alter the board topology and help break repetition equilibria observed in self-play. The higher endgame mate-

rial weight reflects the turn-limit decision rule: a small material lead should be preserved into the final turns rather than spent on positional play.

4.2.2 Threat Differential

For each side, we sum the height of every stack that could be captured by an adjacent enemy stack on the next turn. The signed difference is added to $E(s)$ with coefficient 15. This rewards both attacking the opponent’s vulnerable stacks and defending our own vulnerable stacks in a single term. Empirically, this was the largest single contributor among the evaluation features, as shown in Section 5.3.

4.2.3 Terminal Draw Value

Terminal draws score -100 . This is small enough that losing positions stay worse than draws, but large enough to discourage drifting into forced repetition when winning chances remain.

4.2.4 Tuning Procedure

Coefficient changes were retained only when supported by sufficient match-play evidence. Each candidate setting was evaluated over 20 to 30 games against the depth-2 baseline described in Section 5.1. We accepted a change only when its 95% Wilson confidence interval did not strictly contain the previous setting’s point estimate. Smaller candidate differences were resolved using paired self-play matches.

5 Testing and Evaluation

5.1 Test Setup

We evaluated the agent against three reference opponents:

- **agent_random**: chooses uniformly at random from the legal action set.
- **agent_greedy**: applies each legal action, evaluates the resulting state using our evaluation function, and selects the best action with one-ply lookahead.
- **agent_weak**: uses depth-2 α - β search with a simplified evaluation function containing only material and capture count.

Each matchup ran n games, with our agent playing as Red in half of the games and as Blue in the other half. We compute 95% confidence intervals using the Wilson score method, which is more reliable than the normal approximation for small sample sizes or extreme proportions. We used $n = 30$ for the depth-2 baseline, which is the main benchmark, and $n = 10$ for the weaker reference opponents, which is sufficient to detect catastrophic regressions. At $n = 30$ with an 85% point estimate, the 95% Wilson interval is approximately ± 12 percentage points, allowing reliable detection of effects of about 15 percentage points or more.

5.2 Baseline Performance

Table 5 reports performance against the three reference opponents. Results against random and greedy are perfect, confirming that the search framework handles weak adversaries reliably. The four non-wins against **agent_weak** were all draws, with no losses: two

Opponent	Games	Wins	Losses	Draws	Win rate	95% Wilson CI
<code>agent_random</code>	10	10	0	0	100.0%	[72.2, 100.0]
<code>agent_greedy</code>	10	10	0	0	100.0%	[72.2, 100.0]
<code>agent_weak</code>	30	26	0	4	86.7%	[70.3, 94.7]

Table 5: Baseline performance against reference opponents

were early threefold repetitions and the other two occurred in balanced midgame and late-game positions where neither side could force a material advantage.

5.3 Algorithmic Ablation

To measure the contribution of each design choice, we disabled each component individually and re-measured performance against `agent_weak` with $n = 30$.

Configuration	Wins	Draws	Win rate	95% CI	Δ
Full	26	4	86.7%	[70.3, 94.7]	—
Without transposition table	25	5	83.3%	[66.4, 92.7]	−3.4 pp
Without killer moves	23	5	76.7%	[59.1, 88.2]	−10.0 pp
Without threat term	20	7	66.7%	[48.8, 80.8]	−20.0 pp
Without phase-aware weights	24	5	80.0%	[62.7, 90.5]	−6.7 pp
All four disabled	18	8	60.0%	[42.3, 75.4]	−26.7 pp

Table 6: Ablation results against `agent_weak`

The threat differential is the largest single contributor, accounting for a 20 percentage point improvement. It is also the only ablation whose 95% Wilson interval shows clear separation from the full agent. This effect is mirrored in the draw frequency, which rises from 13.3% to 23.3% when the threat term is disabled. Without this term, the agent identifies fewer capture-or-be-captured opportunities and drifts into passive, material-balanced positions.

The **killer-move heuristic** contributes the second-largest effect, with a 10.0 percentage point reduction when disabled. By promoting actions that caused β -cutoffs at the same depth and side-to-move elsewhere in the search tree, it improves the effectiveness of α - β pruning.

Phase-aware weights contribute a 6.7 percentage point improvement. This effect is within the statistical noise floor at $n = 30$, since the intervals overlap, but the underlying mechanism is consistent with self-play observations: a higher opening stack weight helps disrupt threefold-repetition equilibria.

The transposition table shows the smallest measured effect at 3.4 percentage points, which is within likely sampling noise. A possible explanation is that the agent’s other move-ordering mechanisms already recover much of the same benefit at shallow depths. However, the effect becomes more visible in the per-turn search statistics. Across 354 logged play-phase turns against `agent_weak`, the full agent reached depth 4 or deeper on 4.8% of turns, whereas the configuration without a transposition table never reached depth 4 within the 0.7s search budget (Table 7). This suggests that the primary contribution of the transposition table is enabling deeper searches.

Configuration	Mean depth	% $\geq$ depth 3	% $\geq$ depth 4	Mean turn time
Full	2.98	91.0%	4.8%	0.274 s
No TT	2.85	84.9%	0.0%	0.313 s

Table 7: Per-turn search statistics (Full: 354 play-phase turns; No TT: 232)

Disabling all four components together drops the win rate to 60.0%. This floor reflects the remaining strong components: the custom move-ordering function, iterative deepening with anytime fallback, and the hand-tuned midgame evaluation weights, all of which remain active in the minimal configuration.

5.4 Self-Play Observations

We also ran the agent against itself to characterise its behaviour in symmetric play. Earlier in development, self-play games consistently ended in 38- or 39-turn threefold-repetition draws. Both sides made deterministically equivalent moves into a small repeating position. Raising the opening-phase stack weight from 40 to 60 broke this equilibrium by promoting merges, which reduce the number of stacks on the board and change its topology. After this change, self-play games no longer terminate in short repetition draws. The typical outcome is either a midgame decision around turn 150–250 or a turn-limit conclusion with a material difference.

5.5 Variability and Measurement Limitations

Early experiments showed substantial run-to-run variability in win rate against `agent_weak`, ranging from 95% in one measurement to 65% in another despite identical code. Investigation suggested that CPU contention from other host processes caused iterative deepening to complete fewer plies within the 0.7-second per-turn budget, leading to shallower evaluations and more frequent draws.

The measurements reported above were taken with the host machine under controlled load. Even so, the 95% Wilson intervals at $n = 30$ are wide, approximately ± 12 to ± 15 percentage points. Therefore, ablation effects smaller than about 15 percentage points are difficult to distinguish from sampling noise without a larger sample. Reducing this uncertainty would require at least $n \geq 100$ games per configuration, which exceeded our compute budget given an average game length of approximately 90 seconds.

6 AI Use Acknowledgement

AI tools were used for limited assistance with brainstorming and evaluating algorithmic design alternatives, including the transposition table structure, killer-move indexing strategy, and Zobrist hashing scheme. AI assistance was also used for improving report wording and organising presentation structure.

All core algorithms, implementation details, experiments, evaluations, and final technical decisions were independently developed, verified, and reviewed by the team.